

Módulo 3: Diccionario y FAQ de términos del curso

Glosario práctico para acompañar clases sobre IA, prompting, agentes y gobernanza

Tabla de contenidos

Propósito	2
Cómo leer este diccionario	2
Mapa rápido	2
Diccionario	3
Conceptos base	3
Prompting y calidad	6
Arquitectura de soluciones	8
Operación y despliegue	11
Riesgo, criterio y gobernanza	13
Construcción de soluciones	15
FAQ	16
1. ¿Cuál es la diferencia entre IA, IA generativa y LLM?	16
2. ¿Un LLM y un chatbot son lo mismo?	16
3. ¿Por qué el curso insiste tanto en que el prompt es un contrato?	17
4. ¿Un prompt bueno reemplaza RAG?	17
5. ¿Más contexto siempre mejora la respuesta?	17
6. Si existe RAG, entonces ya no hay alucinaciones?	17
7. ¿Cuándo un chatbot pasa a ser agente?	17
8. ¿Qué aporta una herramienta a un agente?	17
9. ¿Qué diferencia hay entre no-code y código asistido?	17
10. ¿Por qué no basta con que una demo funcione bien una vez?	18
11. ¿Qué se debería versionar en un proyecto de IA generativa?	18
12. ¿Qué miden las evals en agentes?	18

13. ¿Por qué importa tanto human-in-the-loop?	18
14. ¿Cuál es un error común al usar IA para decidir?	18
15. ¿Qué parte del problema es tecnológica y qué parte es de diseño?	18
16. ¿Por qué el caso AFP es tan buen ejemplo pedagógico?	18
Confusiones comunes	19
Términos mínimos que conviene recordar	19
Base del material en este repo	19

Propósito

Este material resume el vocabulario que aparece a lo largo del curso y responde dudas frecuentes de estudiantes. No busca reemplazar las clases, sino ofrecer un lenguaje común para conversar sobre IA generativa, prompting, LLMs, agentes, RAG, MLOps, gobernanza y Decision Science.

Esta versión incorpora contexto de las clases en `qmd` y también de los PDF agregados al repo, especialmente en tres ideas que aparecieron con mucha fuerza:

- el prompt como contrato
- el agente como sistema que percibe, razona y actúa
- el trade-off entre no-code y código asistido al construir soluciones

Cómo leer este diccionario

Cada término importante tiene cinco capas:

- Definición simple: qué significa en lenguaje directo
- Por qué importa: por qué conviene recordarlo en el curso
- Ejemplo del curso: dónde se vio en una actividad o caso
- No confundir con: distinción útil para evitar errores
- Pregunta guía: pregunta corta para discutir o estudiar

Mapa rápido

Bloque	Pregunta central	Términos clave
Conceptos base	Qué es lo que realmente hace un modelo	IA, IA generativa, LLM, token, embedding, contexto

Bloque	Pregunta central	Términos clave
Prompting	Cómo pedir bien y con límites claros	prompt, prompt estructurado, temperatura, CoT, auditable
Arquitectura	Cómo pasa de responder a actuar	agente, herramientas, memoria, workflow, MCP, RAG
Operación	Cómo pasa de demo a sistema confiable	evals, observabilidad, CI/CD, MLOps, trazabilidad
Riesgo y criterio	Cómo usar IA sin delegar juicio ciegamente	guardrails, human-in-the-loop, sesgos, accountability, trade-off
Construcción	Cómo elegir la forma de implementarlo	no-code, código asistido, prototipo, producción

Diccionario

Conceptos base

IA

Definición simple: paraguas amplio para sistemas capaces de clasificar, predecir, recomendar o generar contenido.

Por qué importa: en clase sirve para no meter todo en el mismo saco. No toda IA es conversacional ni toda IA es generativa.

Ejemplo del curso: en `diapositiva_03.qmd` se separa IA de Machine Learning, IA generativa y LLMs.

No confundir con: IA generativa. Esta última es solo una parte del campo.

Pregunta guía: cuando alguien dice “queremos usar IA”, ¿de qué capa está hablando realmente?

Machine Learning

Definición simple: forma de construir IA a partir de datos y aprendizaje de patrones.

Por qué importa: ayuda a distinguir modelos que predicen o clasifican de modelos que conversan o generan texto.

Ejemplo del curso: aparece como contrapunto a los LLMs en las diapositivas introductorias.

No confundir con: LLM. Un LLM es un tipo específico de modelo para lenguaje, no todo Machine Learning.

Pregunta guía: la tarea que queremos resolver es de predicción, clasificación o generación?

IA generativa

Definición simple: familia de sistemas capaces de crear contenido nuevo, como texto, imagen, audio, video o código.

Por qué importa: es la base del curso, pero se trabaja siempre conectada a problemas organizacionales concretos.

Ejemplo del curso: generación de respuestas, informes, PDFs, correos o salidas explicativas en los casos prácticos.

No confundir con: automatización. Generar contenido no implica por sí solo ejecutar un flujo.

Pregunta guía: estamos usando IA para crear contenido o para tomar una acción?

LLM

Definición simple: modelo entrenado con grandes volúmenes de texto para trabajar con lenguaje y producir continuaciones probables dado un contexto.

Por qué importa: es el núcleo de gran parte de las soluciones del curso, pero no equivale al sistema completo.

Ejemplo del curso: en `m03-clase01-arquitectura-llms.qmd` se presenta como capa central del stack; en el caso AFP redacta la respuesta después de recuperar contexto.

No confundir con: chatbot. El LLM es el modelo; el chatbot es una interfaz o aplicación.

Pregunta guía: el problema se resuelve solo con un modelo o necesita también datos, reglas y herramientas?

Token

Definición simple: unidad pequeña de texto con la que opera el modelo.

Por qué importa: costo, longitud de una conversación y límite de contexto se entienden mejor en tokens que en palabras.

Ejemplo del curso: `tokenizacion_tecnicas.qmd` y `diapositiva_03.qmd` muestran que un mismo texto puede ocupar más o menos tokens según el modelo.

No confundir con: palabra. Un token puede ser una palabra, parte de una palabra o un signo.

Pregunta guía: el problema es que el texto es largo o que ocupa demasiados tokens?

Tokenización

Definición simple: proceso de dividir el texto en unidades operativas para el modelo.

Por qué importa: afecta costo, capacidad efectiva de contexto y comportamiento del modelo en distintos idiomas y dominios.

Ejemplo del curso: en `tokenizacion_tecnicas.qmd` se comparan tokenización por caracteres, palabras, subword y bytes.

No confundir con: compresión. Tokenizar no necesariamente acorta.

Pregunta guía: dos textos del mismo largo visual ocupan la misma cantidad de tokens?

Embedding

Definición simple: representación numérica del significado de un texto.

Por qué importa: permite comparar similitud semántica y es pieza central en búsqueda semántica y RAG.

Ejemplo del curso: `diapositiva_03.qmd` y el caso AFP lo conectan con bases vectoriales y recuperación de FAQ o normativa.

No confundir con: respuesta generada. El embedding no responde; ayuda a buscar o representar significado.

Pregunta guía: cuando un sistema “encuentra textos parecidos”, cómo lo está haciendo?

Atención

Definición simple: mecanismo por el cual el modelo decide qué partes del contexto pesan más al responder.

Por qué importa: ayuda a entender por qué no todo lo que ponemos en el prompt tiene la misma influencia.

Ejemplo del curso: `m03-clase01-arquitectura-llms.qmd` lo presenta como intuición para comprender coherencia y relevancia.

No confundir con: memoria perfecta. Que algo esté en el contexto no garantiza que sea priorizado bien.

Pregunta guía: qué partes de la instrucción queremos que el modelo priorice?

Ventana de contexto

Definición simple: cantidad máxima de información que el modelo puede considerar al mismo tiempo.

Por qué importa: determina cuánto cabe de instrucciones, documentos, historial o resultados de herramientas.

Ejemplo del curso: aparece tanto en arquitectura LLM como en agentes con memoria y documentos.

No confundir con: memoria de largo plazo. La ventana de contexto es memoria operativa de la interacción actual.

Pregunta guía: ¿el problema se resuelve con más contexto o con mejor selección de contexto?

Prompting y calidad

Prompt

Definición simple: instrucción que damos al modelo.

Por qué importa: en el curso se insiste en que pedir bien no es un detalle cosmético; cambia radicalmente el resultado.

Ejemplo del curso: el PDF Clase_260406_del-prompt-al-agente (1).pdf muestra que un prompt libre para detectar fraude produjo criterios arbitrarios, mientras un prompt estructurado produjo una salida mas auditable.

No confundir con: pregunta informal. Un prompt puede incluir rol, objetivo, restricciones, formato y ejemplos.

Pregunta guía: si el resultado fue malo, ¿falló el modelo o la instrucción?

Prompt estructurado

Definición simple: prompt que explicita contexto, objetivo, restricciones, audiencia y formato esperado.

Por qué importa: vuelve más predecible la salida y ayuda a que el resultado sea defendible ante otros.

Ejemplo del curso: en la actividad de fraude del PDF Clase_260406_del-prompt-al-agente (1).pdf, el cambio clave fue pasar de prompt libre a prompt estructurado y auditable.

No confundir con: prompt largo. Un prompt puede ser largo y seguir siendo vago.

Pregunta guía: ¿qué tendría que incluir el prompt para que el resultado sea auditable?

Prompt como contrato

Definición simple: idea de que el prompt define con precisión que esperamos del modelo, como si fuera un entregable acordado.

Por qué importa: es uno de los mensajes más fuertes de los PDFs nuevos y ayuda mucho a estudiantes no técnicos.

Ejemplo del curso: el PDF Clase_260406_del-prompt-al-agente (1).pdf resume que “el prompt es el contrato entre tú y el modelo”; el PDF Clase-03-No-Code-vs-Codigo-Asistido.pdf agrega que en soluciones no-code el prompt funciona casi como código del agente.

No confundir con: “hablar bonito”. La idea no es persuadir al modelo, sino delimitar expectativas y errores permitidos.

Pregunta guía: si esto fuera un encargo profesional, ¿el prompt deja claro qué hay que entregar y qué no?

Temperatura

Definición simple: parámetro que modifica cuánta variabilidad tiene la respuesta del modelo.

Por qué importa: en tareas críticas afecta repetibilidad y consistencia.

Ejemplo del curso: el PDF Clase_260406_del-prompt-al-agente (1).pdf muestra el contraste entre $T=0.0$ para auditoría y valores más altos para tareas creativas.

No confundir con: calidad. Más creatividad no significa mejor resultado.

Pregunta guía: ¿esta tarea necesita novedad o necesita reproducibilidad?

Chain of Thought (CoT)

Definición simple: técnica que pide al modelo analizar paso a paso antes de concluir.

Por qué importa: puede ayudar a tareas complejas, siempre que luego exista verificación y no se confunda razonamiento aparente con verdad garantizada.

Ejemplo del curso: en el PDF Clase_260406_del-prompt-al-agente (1).pdf aparece como técnica para analizar fraude con pasos intermedios.

No confundir con: evidencia externa. Pensar paso a paso no reemplaza datos ni fuentes.

Pregunta guía: el problema necesita descomponer el razonamiento o necesita más información?

Alucinación

Definición simple: respuesta inventada, distorsionada o no sustentada, aunque suene convincente.

Por qué importa: es uno de los riesgos más recurrentes del curso, sobre todo cuando falta contexto o se sobreconfía en una salida elegante.

Ejemplo del curso: en la actividad de fraude, el modelo inventa criterios cuando los datos y el prompt no bastan; en AFP el riesgo es responder con seguridad algo sensible sin base suficiente.

No confundir con: error tipográfico o simple imprecisión. La alucinación es un problema de contenido no sustentado.

Pregunta guía: ¿de dónde sale exactamente esta afirmación?

Auditable

Definición simple: resultado cuya lógica, fuentes y criterio pueden explicarse y defenderse ante terceros.

Por qué importa: aparece en varios materiales como exigencia organizacional, especialmente para finanzas, auditoría, riesgo y cumplimiento.

Ejemplo del curso: en la actividad de fraude se pregunta explícitamente si el criterio puede defenderse ante un jefe o un comité de auditoría.

No confundir con: solo “suena profesional”. Una salida pulida puede seguir siendo imposible de auditar.

Pregunta guía: ¿podríamos justificar esta respuesta frente a un comité?

Arquitectura de soluciones

Chatbot

Definición simple: interfaz conversacional que responde mensajes.

Por qué importa: ayuda a no sobrevalorar la capa visible del sistema.

Ejemplo del curso: diapositiva_03.qmd lo contrasta con asistente, copilot y agente.

No confundir con: agente. Un chatbot puede limitarse a responder sin usar herramientas ni ejecutar pasos.

Pregunta guía: ¿el sistema solo conversa o también resuelve tareas?

Asistente

Definición simple: sistema que ayuda a una persona en una tarea.

Por qué importa: pone el foco en apoyo al trabajo humano, no solo en diálogo.

Ejemplo del curso: en varios casos el sistema sugiere, resume o redacta, pero no necesariamente actúa solo.

No confundir con: automatización total. Un asistente puede requerir confirmación constante.

Pregunta guía: estamos aumentando capacidad humana o reemplazando un proceso?

Copilot

Definición simple: asistente integrado dentro de una herramienta o flujo de trabajo.

Por qué importa: muestra una modalidad concreta de adopción organizacional.

Ejemplo del curso: aparece como tipo de sistema distinto a chatbot y agente.

No confundir con: interfaz separada. El copilot acompaña trabajo ya existente dentro de otra herramienta.

Pregunta guía: la ayuda ocurre dentro del flujo de trabajo o en un canal aparte?

Agente

Definición simple: sistema que recibe un objetivo, decide pasos, usa contexto o herramientas y produce una salida.

Por qué importa: es uno de los conceptos centrales del curso y de los PDFs nuevos.

Ejemplo del curso: el caso AFP responde consultas usando fuentes oficiales; el PDF **Agentes-LLMs-Herramientas-que-Actuan.pdf** insiste en que la diferencia clave es que el agente toma decisiones en el camino y no solo devuelve una respuesta.

No confundir con: prompt único. Un agente puede usar múltiples turnos, herramientas, memoria y controles.

Pregunta guía: ¿el sistema solo responde o decide cómo avanzar?

Herramientas

Definición simple: capacidades externas que el agente puede usar, como leer archivos, consultar SQL, ejecutar código, buscar en la web, generar PDF o enviar correo.

Por qué importa: los PDFs nuevos las describen como “el músculo del agente”.

Ejemplo del curso: **Agentes-LLMs-Herramientas-que-Actuan.pdf** enumera categorías como datos, cómputo, búsqueda, escritura, memoria y control; **Clase-03-No-Code-vs-Codigo-Asistido.pdf** muestra un agente que compara viajes, genera PDF y lo envía por correo.

No confundir con: conocimiento interno del LLM. Una herramienta trae capacidad operativa o acceso a información externa.

Pregunta guía: ¿qué herramienta concreta falta para que este agente sea realmente útil?

Workflow

Definición simple: secuencia de pasos mediante la cual una solución transforma una entrada en una salida.

Por qué importa: evita pensar el sistema solo como una respuesta aislada.

Ejemplo del curso: en AFP el flujo es consulta, recuperación de información, redacción, control y entrega; en viajes el flujo busca vuelos, hoteles, atracciones, calcula presupuesto y genera un informe.

No confundir con: prompt. El prompt es una pieza; el workflow es el proceso completo.

Pregunta guía: ¿cuáles son los pasos reales desde la pregunta hasta la acción final?

Memoria

Definición simple: mecanismos para retener contexto entre pasos, sesiones o documentos.

Por qué importa: en agentes cambia mucho la experiencia y la capacidad de continuidad.

Ejemplo del curso: `Agentes-LLMs-Herramientas-que-Actuan.pdf` distingue memoria de contexto, caché, vectorial y estructurada.

No confundir con: ventana de contexto. Memoria puede ir más allá de la conversación actual.

Pregunta guía: ¿qué necesita recordar el sistema y por cuánto tiempo?

RAG

Definición simple: forma de responder usando información recuperada desde documentos o bases externas.

Por qué importa: reduce dependencia de la memoria genérica del modelo y mejora trazabilidad.

Ejemplo del curso: en AFP se usa para responder con FAQ, cartola y normativa; en `Agentes-LLMs-Herramientas-que-Actuan.pdf` se ejemplifica con Ley de Renta y documentos del SII.

No confundir con: fine-tuning. RAG trae contexto al momento de responder; no reentrena el modelo.

Pregunta guía: ¿de dónde saldrá la información específica y actualizada que necesita el sistema?

Base de datos vectorial

Definición simple: infraestructura pensada para guardar y buscar embeddings.

Por qué importa: suele ser la pieza técnica que vuelve operativo un esquema RAG.

Ejemplo del curso: aparece asociada a FAQ institucionales, normativa y documentos internos.

No confundir con: base relacional tradicional. Una sirve para similitud semántica; la otra para consultas estructuradas.

Pregunta guía: ¿necesito buscar por significado o por campos exactos?

MCP

Definición simple: protocolo estándar para conectar agentes o modelos con herramientas externas.

Por qué importa: es uno de los términos nuevos que aparece en los materiales recientes y ayuda a explicar integración sin bajar demasiado a detalle técnico.

Ejemplo del curso: `Agentes-LLMs-Herramientas-que-Actuan.pdf` lo presenta como el estándar que hace a los agentes más interoperables, comparándolo con USB para periféricos.

No confundir con: una herramienta específica. MCP no es la herramienta; es la forma estandarizada de conectar herramientas.

Pregunta guía: ¿la integración está hecha a mano o sigue un estándar reusable?

Multi-agente

Definición simple: arquitectura donde varios agentes especializados colaboran para resolver una tarea.

Por qué importa: aparece como respuesta a problemas con subtareas, verificación o paralelismo.

Ejemplo del curso: `Agentes-LLMs-Herramientas-que-Actuan.pdf` muestra un equipo de cobranza automatizada con orquestador, analista, buscador normativo, redactor y revisor.

No confundir con: complejidad innecesaria. No todo problema necesita varios agentes.

Pregunta guía: ¿un solo agente basta o conviene separar funciones especializadas?

Operación y despliegue

Evals

Definición simple: pruebas para medir si una solución de IA funciona bien.

Por qué importa: permiten comparar versiones y evitar que la calidad dependa de intuición o anécdotas.

Ejemplo del curso: `m03-clase02-cicd-mlops-gobernanza.qmd` y el PDF de agentes recalcan métricas como task completion, pasos, tool accuracy, hallucination, latencia y costo.

No confundir con: opinión del usuario solamente. El feedback ayuda, pero no reemplaza medición sistemática.

Pregunta guía: ¿cómo sabremos si esta nueva versión realmente mejora?

Observabilidad

Definición simple: capacidad de mirar cómo se comporta el sistema en uso real.

Por qué importa: un sistema puede seguir “funcionando” y al mismo tiempo estar respondiendo peor, costando más o fallando de formas silenciosas.

Ejemplo del curso: calidad, latencia, costo, uso de contexto, fallos de herramientas y feedback del usuario.

No confundir con: monitoreo técnico mínimo. En IA generativa hay que observar también calidad semántica.

Pregunta guía: ¿qué veríamos en los datos si el sistema empieza a degradarse?

CI/CD

Definición simple: prácticas para integrar cambios frecuentes y desplegarlos con más confiabilidad.

Por qué importa: en IA generativa se versionan no solo archivos de código, sino también prompts, configuraciones, datasets de prueba y evaluaciones.

Ejemplo del curso: `m03-clase02-cicd-mlops-gobernanza.qmd` lo presenta como paso de experimentación a despliegue continuo.

No confundir con: solo automatizar deploy. Incluye integración, pruebas y validación de cambios.

Pregunta guía: ¿qué cambia realmente cuando modificamos el sistema?

MLOps

Definición simple: conjunto de prácticas para desarrollar, desplegar, operar y mejorar sistemas basados en modelos de manera confiable.

Por qué importa: conecta datos, modelos, ingeniería y operación.

Ejemplo del curso: en clase se relaciona con drift, monitoreo, evaluación, observabilidad y mejora continua.

No confundir con: DevOps puro. MLOps agrega problemas propios de modelos, datos y evaluaciones no deterministas.

Pregunta guía: ¿qué hace falta para que esto deje de ser demo y pase a capacidad organizacional?

Drift

Definición simple: deterioro o cambio del comportamiento del sistema a lo largo del tiempo.

Por qué importa: explica por qué una solución puede empeorar sin que “se rompa” técnicamente.

Ejemplo del curso: cambios en datos, usuarios, prompts, modelos del proveedor o procesos internos.

No confundir con: bug único. El drift suele ser gradual y acumulativo.

Pregunta guía: si esto funcionaba bien hace dos meses, ¿qué pudo haber cambiado?

Trazabilidad

Definición simple: capacidad de reconstruir qué información se usó, qué reglas aplicaron y cómo se llegó a un resultado.

Por qué importa: es clave para auditoría, seguridad, gobernanza y mantenimiento.

Ejemplo del curso: los PDFs nuevos insisten en lógica auditable, herramientas visibles y código modificable; el de no-code vs código asistido advierte que sin logs ni trazabilidad el sistema es imposible de auditar.

No confundir con: tener solo el output final. La trazabilidad incluye el camino.

Pregunta guía: ¿podríamos explicar cómo se llegó a esta respuesta o acción?

Riesgo, criterio y gobernanza

Gobernanza

Definición simple: conjunto de reglas, roles y controles para asegurar uso responsable y alineado con objetivos organizacionales.

Por qué importa: ayuda a decidir qué se permite, quién responde y qué riesgos se aceptan.

Ejemplo del curso: `m03-clase02-cicd-mlops-gobernanza.qmd` la conecta con privacidad, seguridad, roles, políticas y control.

No confundir con: prohibición. Gobernar no es impedir, sino ordenar y responsabilizar.

Pregunta guía: ¿quién decide los límites de este sistema y bajo qué reglas?

Guardrails

Definición simple: controles para limitar comportamientos no deseados.

Por qué importa: en agentes son esenciales para restringir temas, herramientas, acciones y escalamiento.

Ejemplo del curso: en AFP sirve para impedir recomendaciones previsionales personalizadas sin revisión; en agentes con herramientas ayuda a evitar acciones irreversibles o alcance excesivo.

No confundir con: buen prompt solamente. Los guardrails pueden vivir también en reglas del sistema, permisos y aprobaciones.

Pregunta guía: ¿qué no queremos que este agente haga nunca por su cuenta?

Human-in-the-loop

Definición simple: esquema donde una persona revisa, corrige o aprueba parte del proceso.

Por qué importa: es la forma práctica de graduar autonomía según riesgo.

Ejemplo del curso: `Agentes-LLMs-Herramientas-que-Actuan.pdf` define niveles de autonomía según el tipo de acción; para decisiones de cobranza recomienda aprobación o reporte de excepciones.

No confundir con: supervisión constante de todo. El punto es decidir bien en qué momentos humanos deben intervenir.

Pregunta guía: ¿qué acción requiere aprobación humana antes de ejecutarse?

Trade-off

Definición simple: tensión entre objetivos que no pueden maximizarse al mismo tiempo.

Por qué importa: aparece transversalmente en el curso, desde precisión versus velocidad hasta control versus facilidad de implementación.

Ejemplo del curso: AFP trabaja velocidad versus precisión y autonomía versus control; el PDF de no-code versus código asistido trabaja velocidad y facilidad versus control y auditabilidad.

No confundir con: error de diseño. Algunos trade-offs son inevitables; lo importante es explicitarlos.

Pregunta guía: ¿qué estamos ganando y qué estamos sacrificando con esta decisión?

Decision Science

Definición simple: disciplina que ayuda a estructurar decisiones complejas bajo incertidumbre.

Por qué importa: instala la idea de partir por el problema y no por la tecnología.

Ejemplo del curso: `m03-clase03-decision-science-sesgos.qmd` y el caso AFP preguntan primero qué decisión organizacional estamos tratando de mejorar.

No confundir con: analítica técnica solamente. Incluye criterios, consecuencias, incertidumbre y responsables.

Pregunta guía: ¿qué decisión real queremos mejorar con este sistema?

Sesgo de automatización

Definición simple: tendencia a confiar demasiado en sistemas automáticos.

Por qué importa: es uno de los riesgos humanos más relevantes al usar IA en organizaciones.

Ejemplo del curso: en AFP se advierte el riesgo de tomar una respuesta bien escrita como recomendación válida; en decision science se presenta como una señal de alerta organizacional.

No confundir con: eficiencia. Que un sistema ahorre tiempo no significa que deba recibir confianza ciega.

Pregunta guía: ¿estamos revisando la salida o solo aceptándola porque viene del sistema?

Accountability

Definición simple: responsabilidad por las decisiones y consecuencias del uso de IA.

Por qué importa: recuerda que la responsabilidad institucional no desaparece porque participe una herramienta.

Ejemplo del curso: clase 3 de decision science lo vincula con criterio profesional y responsabilidad humana.

No confundir con: culpa del proveedor o del modelo. La organización sigue respondiendo por su uso.

Pregunta guía: si esto sale mal, ¿quién responde y con qué evidencia?

Construcción de soluciones

No-code

Definición simple: forma de construir prototipos o soluciones usando interfaces visuales o descripciones en lenguaje natural, con mínima programación directa.

Por qué importa: baja la barrera de entrada y acelera el prototipado.

Ejemplo del curso: el PDF Clase-03-No-Code-vs-Codigo-Asistido.pdf muestra v0.app, n8n y otras herramientas para construir rápido un agente funcional.

No confundir con: producción segura por defecto. Rapidez de prototipo no implica mantenibilidad ni auditabilidad suficientes.

Pregunta guía: ¿estamos validando una idea rápido o construyendo algo para operar de verdad?

Código asistido

Definición simple: forma de construir software donde una IA ayuda a escribir, ejecutar y corregir código dentro de un proyecto real.

Por qué importa: permite más control, integraciones y trazabilidad, pero exige más criterio técnico.

Ejemplo del curso: Clase-03-No-Code-vs-Codigo-Asistido.pdf usa Claude Code para construir un agente comparador de viajes que genera PDF y envía correo.

No confundir con: programación totalmente automática. Sigue requiriendo arquitectura, pruebas, seguridad y mantenimiento.

Pregunta guía: ¿necesitamos velocidad de prototipo o control fino sobre el sistema?

Prototipo

Definición simple: versión inicial para validar una idea rápidamente.

Por qué importa: los PDFs nuevos insisten en que un prototipo visible puede impresionar mucho, pero no equivale a producto listo.

Ejemplo del curso: el agente de viajes y varias actividades están pensados para salir de clase con algo funcional.

No confundir con: producción. Un prototipo puede carecer de pruebas, logs, seguridad o manejo de errores.

Pregunta guía: ¿estamos validando una hipótesis o prometiendo operación real?

Producción

Definición simple: etapa en que el sistema opera con usuarios reales, datos reales y consecuencias reales.

Por qué importa: cambia el estándar exigido: seguridad, trazabilidad, monitoreo, manejo de errores, costos y mantenimiento.

Ejemplo del curso: Clase-03-No-Code-vs-Codigo-Asistido.pdf recalca que pasar de prototipo a producción sigue requiriendo ingeniería de software.

No confundir con: demo exitosa. Una demo muestra posibilidad; la producción exige confiabilidad sostenida.

Pregunta guía: ¿qué nos falta para poner esto frente a usuarios reales con tranquilidad?

FAQ

1. ¿Cuál es la diferencia entre IA, IA generativa y LLM?

IA es el paraguas amplio. IA generativa es la parte que crea contenido nuevo. Un LLM es un tipo de modelo generativo especializado en lenguaje.

2. ¿Un LLM y un chatbot son lo mismo?

No. El LLM es el modelo. El chatbot es una interfaz o aplicación que puede usar un LLM, documentos, reglas, memoria y herramientas.

3. ¿Por qué el curso insiste tanto en que el prompt es un contrato?

Porque una instrucción vaga deja demasiado espacio a que el modelo complete vacíos con patrones genéricos. Los PDFs nuevos muestran que cuando el entregable, criterio y restricciones quedan mejor definidos, la salida se vuelve más útil y más auditable.

4. ¿Un prompt bueno reemplaza RAG?

No. Un buen prompt ayuda a pedir mejor. RAG ayuda a traer información específica y actualizada. Cumplen funciones distintas.

5. ¿Más contexto siempre mejora la respuesta?

No. A veces mejora; otras veces ensucia, distrae o encarece. Muchas veces importa más seleccionar bien el contexto que simplemente agregar más.

6. Si existe RAG, entonces ya no hay alucinaciones?

No. RAG reduce parte del problema, pero no elimina errores de recuperación, interpretación o redacción.

7. ¿Cuándo un chatbot pasa a ser agente?

Cuando deja de limitarse a responder y empieza a decidir pasos, usar herramientas, trabajar con objetivos y ejecutar acciones o subprocesos.

8. ¿Qué aporta una herramienta a un agente?

Le da capacidad operativa o acceso a información externa. Sin herramientas, el agente puede razonar sobre texto; con herramientas puede buscar, calcular, leer, escribir, enviar o consultar sistemas.

9. ¿Qué diferencia hay entre no-code y código asistido?

No-code privilegia velocidad y baja barrera de entrada. Código asistido privilegia control, integración y auditabilidad. Ambos pueden servir, pero no para los mismos contextos.

10. ¿Por qué no basta con que una demo funcione bien una vez?

Porque un sistema real debe sostener calidad, seguridad, costo razonable, monitoreo, trazabilidad y mantenimiento en el tiempo.

11. ¿Qué se debería versionar en un proyecto de IA generativa?

Código, prompts, configuraciones del modelo, datasets de prueba, reglas de negocio y resultados de evaluación.

12. ¿Qué miden las evals en agentes?

No solo calidad del texto final. También completion rate, cantidad de pasos, precisión en el uso de herramientas, alucinación, latencia y costo por tarea.

13. ¿Por qué importa tanto human-in-the-loop?

Porque hay acciones y decisiones que no deberían delegarse ciegamente, especialmente cuando son irreversibles o tienen impacto financiero, legal o humano.

14. ¿Cuál es un error común al usar IA para decidir?

Pedirle a la IA que diga “qué hacer” como si reemplazara criterio profesional. Suele ser mejor pedir alternativas, contraargumentos, evidencia usada, vacíos de información y riesgos no considerados.

15. ¿Qué parte del problema es tecnológica y qué parte es de diseño?

La tecnología resuelve acceso a modelos, herramientas y automatización. El diseño define alcance, fuentes, riesgo aceptable, reglas, escalamiento y criterios de evaluación. Sin buen diseño, la tecnología sola no salva la solución.

16. ¿Por qué el caso AFP es tan buen ejemplo pedagógico?

Porque obliga a pensar en fuentes oficiales, claridad, riesgo, escalamiento a humano, trazabilidad y límites de autonomía. Muestra muy bien que “responder” no es suficiente.

Confusiones comunes

- LLM no es lo mismo que chatbot.
- prompt no es solo una pregunta; puede ser una especificación completa.
- RAG no es fine-tuning.
- tener muchos datos no significa tener los datos correctos.
- un prototipo funcional no es un sistema listo para producción.
- no-code no elimina la necesidad de diseño, pruebas y gobernanza.

Términos mínimos que conviene recordar

- LLM
- contexto
- prompt estructurado
- temperatura
- RAG
- agente
- herramienta
- memoria
- evals
- trazabilidad
- guardrails
- human-in-the-loop
- trade-off

Base del material en este repo

Este diccionario y FAQ se construyó a partir del vocabulario y enfoque presentes en:

- diapositiva_03.qmd
- m03-clase01-arquitectura-llms.qmd
- m03-clase02-cicd-mlops-gobernanza.qmd
- m03-clase03-decision-science-sesgos.qmd
- m03-caso-aplicado-agente-afp.qmd
- tokenizacion_tecnicas.qmd
- materiales/Clase_260406_del-prompt-al-agente (1).pdf
- materiales/Agentes-LLMs-Herramientas-que-Actuan.pdf
- materiales/Clase-03-No-Code-vs-Codigo-Asistido.pdf